PROJETO INTEGRADOR
MODELAGEM DE BANCO DE DADOS

Modelagem de ERP para uma Micro e Pequena Empresa

Manual de desenvolvimento passo a passo para as equipes

Disciplina: Modelagem de Banco de Dados
Professor: Gabriel Rosa Paz

Visão do projeto
O projeto deve partir de um problema empresarial real ou realisticamente documentado e evoluir, de forma rastreável, até um banco de dados relacional implementado, testado, documentado e defendido tecnicamente. A equipe não deve começar pelo SQL: primeiro deve compreender a empresa, seus processos, dores, requisitos e regras de negócio.

1. Objetivo geral do projeto

Ao final da disciplina, os estudantes deverão ser capazes de analisar, modelar e documentar os requisitos de um sistema ERP para micro e pequenas empresas, aplicando técnicas de engenharia de requisitos, modelagem conceitual, modelagem lógica, normalização e implementação de banco de dados. O desenvolvimento deve ocorrer de forma progressiva, colaborativa e fundamentada em situações reais do ambiente empresarial.

1.1 Competências desenvolvidas

1. Levantar requisitos e regras de negócio.
2. Elaborar modelos conceituais.
3. Aplicar normalização.
4. Transformar modelos conceituais em modelos lógicos.
5. Implementar e validar estruturas de banco de dados.
6. Avaliar soluções propostas.
7. Produzir documentação técnica e apresentar soluções.
8. Converter modelos conceituais em estruturas relacionais normalizadas.
9. Projetar bases de dados relacionais consistentes.
10. Implementar mecanismos de integridade e consultas SQL.
11. Produzir documentação técnica corporativa.
12. Defender tecnicamente soluções de banco de dados.
13. Aplicar conceitos de governança e análise de dados.

1.2 O que significa modelar um ERP?

ERP (Enterprise Resource Planning) é um sistema integrado de gestão. Para este projeto, a equipe não precisa construir uma aplicação ERP completa com telas e todas as funcionalidades de um produto comercial. O foco da disciplina é projetar e implementar a base de dados que sustentaria um ERP simples para uma micro ou pequena empresa.

- Cadastro de clientes, fornecedores, funcionários e usuários.

- Produtos, serviços, categorias e estoque.
- Compras, vendas, pedidos, itens e pagamentos.
- Contas a pagar e a receber, quando fizer sentido para o negócio.
- Auditoria, histórico e dados gerenciais.
- Informações que possam futuramente alimentar relatórios, BI ou soluções de IA.

Importante
A equipe deve escolher somente os módulos necessários para representar adequadamente a empresa estudada. Um projeto menor, coerente e bem justificado é melhor do que um grande número de tabelas criadas sem relação clara com os requisitos.

2. Princípio metodológico do projeto

O projeto deverá seguir uma linha de rastreabilidade. Toda decisão técnica precisa ter origem identificável em um problema, processo, requisito ou regra de negócio.

Fluxo recomendado:

```
EMPRESA -> PROCESSOS -> PROBLEMAS -> REQUISITOS -> REGRAS DE NEGÓCIO ->
ENTIDADES -> ATRIBUTOS -> RELACIONAMENTOS -> CARDINALIDADES -> DER ->
NORMALIZAÇÃO -> MODELO LÓGICO -> MODELO FÍSICO -> SQL -> TESTES -> BI/IA
```

Regra de ouro
Durante a apresentação, qualquer integrante deve conseguir responder: "Por que esta tabela, atributo, relacionamento ou restrição existe e de qual requisito/regra de negócio veio esta decisão?"

3. Escolha da empresa e definição do escopo

3.1 Escolha da empresa

A equipe deverá selecionar uma micro ou pequena empresa real, uma empresa conhecida pelos integrantes ou um cenário empresarial suficientemente documentado. O importante é que seja possível compreender como o negócio opera.

- Lanchonete, cafeteria ou restaurante.
- Loja de roupas, calçados ou eletrônicos.
- Assistência técnica.
- Clínica, consultório ou laboratório.
- Academia ou estúdio.
- Papelaria, mercado, pet shop ou loja de bairro.
- Prestadora de serviços, oficina, gráfica ou pequena indústria.
- E-commerce de pequeno porte.

3.2 Caracterização mínima

Item	O que documentar
Empresa	Nome fantasia ou nome fictício coerente; indicar quando for cenário simulado.
Porte	Microempresa ou pequena empresa.
Segmento	Área de atuação e contexto de mercado.
Produtos/serviços	O que a empresa vende ou oferece.

Item	O que documentar
Público-alvo	Principais clientes atendidos.
Processos principais	Ex.: vendas, compras, estoque, atendimento, cobrança, entrega.
Problemas atuais	Falhas, retrabalho, falta de controle, inconsistências ou ausência de informação gerencial.

3.3 Definição do escopo

Antes de modelar, a equipe deve escrever claramente o que estará dentro e fora do projeto. O escopo evita que o ERP cresça indefinidamente.

```
Exemplo de escopo:
INCLUÍDO: cadastro de clientes, produtos, fornecedores, compras, vendas, estoque e
pagamentos.
FORA DO ESCOPO: folha de pagamento, emissão fiscal oficial, integração bancária e aplicativo
móvel.
```

4. Fase 1 - Levantamento do negócio, requisitos e regras

4.1 Mapeamento dos processos

A equipe deverá descrever como o trabalho acontece atualmente. Não é necessário criar fluxogramas complexos no início; uma descrição passo a passo já é suficiente para descobrir os dados que o sistema precisará armazenar.

```
Exemplo - Processo de venda:
1. Cliente solicita produtos.
2. Atendente identifica ou cadastra o cliente.
3. Produtos são adicionados ao pedido.
4. Sistema calcula o valor.
5. Pagamento é registrado.
6. Estoque é atualizado.
7. Venda é finalizada.
```

4.2 Dores e problemas operacionais

Registre pelo menos três problemas concretos. Evite frases genéricas como “a empresa precisa melhorar”.

Problema	Consequência	Dado/controle necessário
Estoque divergente	Venda de item indisponível	Movimentações e saldo de estoque.
Clientes duplicados	Histórico fragmentado	Identificador único e validações cadastrais.
Pagamentos sem vínculo claro	Dificuldade de conciliação	Relacionar pagamento à venda/pedido.

4.3 Requisitos funcionais (RF)

Requisitos funcionais descrevem o que o sistema deverá fazer. Cada requisito deve ser numerado e escrito com clareza.

```
RF01 - O sistema deve permitir cadastrar clientes.
RF02 - O sistema deve registrar pedidos e seus itens.
RF03 - O sistema deve registrar entradas e saídas de estoque.
RF04 - O sistema deve permitir consultar vendas por período.
```

4.4 Requisitos não funcionais (RNF)

Requisitos não funcionais descrevem qualidades, restrições e condições de operação.

```
RNF01 - O sistema deve exigir autenticação para usuários administrativos.
RNF02 - O banco deve manter integridade referencial entre pedidos e clientes.
RNF03 - As operações críticas devem manter registro de data/hora e usuário responsável.
```

4.5 Regras de negócio (RN)

As regras de negócio representam políticas e restrições do funcionamento da empresa. Elas devem ser específicas e verificáveis.

```
RN01 - Todo pedido finalizado deve possuir ao menos um item.
RN02 - Um produto inativo não pode ser incluído em novas vendas.
RN03 - O preço praticado no momento da venda deve permanecer registrado no item da venda.
RN04 - Um pagamento deve estar associado a uma venda existente.
```

4.6 Matriz de rastreabilidade

Crie uma tabela relacionando problemas, requisitos, regras e elementos do modelo. Essa matriz será muito útil na defesa técnica.

Problema	RF/RNF	RN	Elemento do modelo
Estoque divergente	RF03	RN - saldo não negativo, se adotada	PRODUTO / MOV_ESTOQUE
Histórico de preço	RF02	RN03	ITEM_VENDA.preco_praticado

5. Fase 2 - Modelo conceitual (MER/DER)

5.1 Identificação das entidades

A partir dos requisitos, identifique os elementos sobre os quais o sistema precisa armazenar informações de forma independente.

- CLIENTE, FORNECEDOR e FUNCIONARIO representam pessoas ou organizações.
- PRODUTO e SERVICO representam itens comercializados.
- PEDIDO, VENDA, COMPRA e PAGAMENTO representam eventos/transações.
- ITEM_PEDIDO, ITEM_VENDA ou ITEM_COMPRA podem atuar como entidades associativas.

5.2 Atributos e identificadores

Para cada entidade, liste somente os atributos necessários ao escopo. Indique um identificador conceitual.

Entidade	Atributo	Função	Observação
CLIENTE	id_cliente	Identificador	PK conceitual.
CLIENTE	nome_cliente	Descrição	Obrigatório.
CLIENTE	email	Contato	Definir se será único.
VENDA	id_venda	Identificador	PK conceitual.
VENDA	data_hora	Registro temporal	Importante para auditoria.

5.3 Relacionamentos e cardinalidades

Transforme os relacionamentos em frases de negócio e leia cada relação nos dois sentidos.

```
CLIENTE (0,N) realiza (1,1) VENDA
VENDA (1,N) contém (1,1) ITEM_VENDA
PRODUTO (0,N) compõe (1,1) ITEM_VENDA
```

A cardinalidade mínima representa obrigatoriedade (0 ou 1). A cardinalidade máxima representa multiplicidade (1 ou N).

5.4 Tratamento de N:N

Relacionamentos muitos-para-muitos devem ser analisados cuidadosamente. Em bancos relacionais, normalmente serão convertidos em uma entidade/tabela associativa.

```
Antes:
VENDA N:N PRODUTO

Depois:
VENDA 1:N ITEM_VENDA N:1 PRODUTO
```

Atributos que pertencem ao vínculo, como quantidade e preço praticado, devem ficar na entidade associativa quando dependem daquela ocorrência da relação.

5.5 Dicionário de dados conceitual

Crie uma tabela completa para todas as entidades e atributos.

Entidade	Atributo	Tipo conceitual	Obrigatório?	Identificador?	Descrição
CLIENTE	nome_cliente	Texto(100)	Sim	Não	Nome completo ou razão social do cliente.

Checkpoint do modelo conceitual

Nenhuma entidade, atributo ou cardinalidade deve existir sem justificativa. Antes de avançar, faça o teste: 'Qual requisito ou regra de negócio originou este elemento?'

6. Fase 3 - Normalização (1FN, 2FN e 3FN)

A normalização tem como objetivo reduzir redundância e dependências inadequadas, evitando anomalias de inserção, atualização e exclusão.

6.1 Antes de normalizar

Escolha uma estrutura representativa do projeto e documente sua evolução. Não basta entregar tabelas finais: o relatório deve mostrar o raciocínio.

```
Exemplo inicial não normalizado:
VENDA(id_venda, data, cliente_nome, cliente_telefone,
      produto1, qtd1, preco1, produto2, qtd2, preco2, ...)
```

6.2 Primeira Forma Normal - 1FN

- Cada campo deve conter um valor atômico adequado ao modelo.
- Evitar grupos repetitivos como produto1, produto2, produto3.
- Cada linha precisa representar uma ocorrência identificável.

```
VENDA(id_venda, data, id_cliente)
ITEM_VENDA(id_venda, id_produto, quantidade, preco_praticado)
```

6.3 Segunda Forma Normal - 2FN

A 2FN é especialmente importante quando existe chave composta. Todo atributo não-chave deve depender da chave inteira, e não apenas de parte dela.

```
Se ITEM_VENDA tem PK (id_venda, id_produto):
quantidade e preco_praticado dependem da combinação venda + produto.
nome_produto não deve ficar em ITEM_VENDA, pois depende apenas de id_produto.
```

6.4 Terceira Forma Normal - 3FN

Na 3FN, atributos não-chave não devem depender de outros atributos não-chave. A estrutura deve evitar dependências transitivas desnecessárias.

```
Evitar:
CLIENTE(id_cliente, nome, id_cidade, nome_cidade, uf)

Possível separação, se o escopo justificar:
CLIENTE(id_cliente, nome, id_cidade)
CIDADE(id_cidade, nome_cidade, uf)
```

6.5 Quadro obrigatório de normalização

Etapa	Problema encontrado	Alteração realizada	Justificativa
1FN	Grupos repetitivos	Criação de ITEM_VENDA	Um item por ocorrência.
2FN	Atributo depende só de parte da chave	Mover nome_produto para PRODUTO	Dependência funcional.
3FN	Dependência transitiva	Separar entidade, se necessário	Evitar redundância e inconsistência.

7. Fase 4 - Conversão para o modelo lógico relacional

O modelo lógico transforma as entidades e relacionamentos do DER em tabelas, atributos, chaves primárias, chaves estrangeiras e restrições.

7.1 Regras de conversão

- Cada entidade principal normalmente origina uma tabela.
- O identificador da entidade torna-se chave primária (PK).
- Relacionamentos 1:N normalmente colocam a PK do lado 1 como FK no lado N.
- Relacionamentos N:N normalmente originam uma tabela associativa.
- Relacionamentos 1:1 exigem análise de obrigatoriedade e dependência.
- Atributos derivados devem ser avaliados antes de serem armazenados.
- Atributos multivalorados geralmente exigem nova tabela.

7.2 Modelo lógico - exemplo

Tabela	Campo	Tipo lógico	Chave	Restrições
CLIENTE	id_cliente	INTEGER	PK	NOT NULL
CLIENTE	nome_cliente	VARCHAR(100)		NOT NULL
VENDA	id_venda	INTEGER	PK	NOT NULL
VENDA	id_cliente	INTEGER	FK	NOT NULL
ITEM_VENDA	id_venda	INTEGER	PK/FK	NOT NULL
ITEM_VENDA	id_produto	INTEGER	PK/FK	NOT NULL
ITEM_VENDA	quantidade	INTEGER		CHECK > 0

7.3 Restrições de integridade

- PRIMARY KEY: identifica unicamente registros.
- FOREIGN KEY: mantém referências válidas entre tabelas.
- NOT NULL: impede ausência de valor quando o dado é obrigatório.
- UNIQUE: impede duplicidade em campos que devem ser exclusivos.
- CHECK: restringe valores permitidos, quando suportado e adequado.

- DEFAULT: define valor padrão quando fizer sentido.

8. Fase 5 - Implementação do banco e Script SQL

O script deve permitir recriar a estrutura e executar testes. Organize o arquivo em seções comentadas.

```
-- 01. Criação do banco
-- 02. Seleção do banco/schema
-- 03. Criação das tabelas
-- 04. Constraints e índices
-- 05. Inserção da massa de dados
-- 06. Consultas de validação
-- 07. Consultas gerenciais/auditoria
-- 08. Exemplos de UPDATE e DELETE
```

8.1 CREATE DATABASE

Se o SGBD escolhido suportar o comando CREATE DATABASE, ele deve aparecer no script conforme solicitado na entrega.

```
CREATE DATABASE erp_empresa;
-- Em seguida, selecione/conecte-se ao banco conforme o SGBD.
```

Observação sobre SQLite
SQLite não utiliza CREATE DATABASE: o banco é criado ao abrir um arquivo .db. Se a equipe optar por SQLite, essa diferença deve ser explicitamente documentada e previamente validada com o professor. Como a entrega exige CREATE DATABASE, MySQL ou PostgreSQL tendem a atender de forma mais direta a esse requisito.

8.2 CREATE TABLE e constraints

```
CREATE TABLE cliente (
    id_cliente INTEGER PRIMARY KEY,
    nome_cliente VARCHAR(100) NOT NULL,
    email VARCHAR(120) UNIQUE
);

CREATE TABLE venda (
    id_venda INTEGER PRIMARY KEY,
    data_hora TIMESTAMP NOT NULL,
    id_cliente INTEGER NOT NULL,
    CONSTRAINT fk_venda_cliente
        FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
);
```

8.3 INSERT INTO

```
INSERT INTO cliente (id_cliente, nome_cliente, email)
VALUES (1, 'Ana Souza', 'ana@exemplo.com');
```

8.4 SELECT com JOIN

```
SELECT
    v.id_venda,
    v.data_hora,
    c.nome_cliente
FROM venda v
INNER JOIN cliente c ON c.id_cliente = v.id_cliente
ORDER BY v.data_hora DESC;
```

8.5 UPDATE e DELETE

```
UPDATE produto
SET preco_atual = 39.90
WHERE id_produto = 10;

DELETE FROM cliente
WHERE id_cliente = 99;
```

O relatório deve explicar riscos e regras para exclusão. Em sistemas reais, apagar dados de histórico pode ser inadequado; em muitos casos usa-se inativação ou exclusão lógica.

8.6 Consultas gerenciais

Crie consultas que respondam perguntas úteis à gestão, por exemplo:

- Faturamento por mês.
- Produtos mais vendidos.
- Clientes com maior volume de compras.
- Estoque atual ou produtos com saldo baixo.
- Compras por fornecedor.
- Vendas por funcionário/atendente, se existir no modelo.
- Ticket médio ou quantidade média de itens por venda.

8.7 Consultas de auditoria

Crie consultas que ajudem a encontrar inconsistências ou situações que merecem revisão:

- Registros sem associação esperada.
- Dados duplicados quando deveriam ser únicos.
- Estoque negativo, se essa situação for proibida.
- Vendas canceladas que possuem pagamento ativo, se aplicável.
- Registros alterados ou transações fora de padrões esperados, se houver campos de auditoria.

9. Fase 6 - Massa de dados para validação

O banco deverá conter registros coerentes e em volume suficiente para validar relacionamentos, regras, consultas e relatórios.

9.1 Qualidade antes de quantidade

- Todas as FKs devem apontar para registros existentes.
- Datas precisam ser coerentes.
- Valores monetários devem ser plausíveis.
- Status devem respeitar as opções definidas.
- Os dados devem permitir testar cenários normais e exceções.

9.2 Uso de LLMs para geração da massa de dados

A equipe poderá utilizar modelos de linguagem para apoiar a geração em escala da massa de dados. Os prompts utilizados devem ser documentados.

Prompt	Objetivo	Saída gerada	Validação humana
Gerar 100 clientes fictícios em CSV...	Popular CLIENTE	100 linhas CSV	Revisão de e-mails, duplicidades e codificação.

Cuidados com IA generativa
Não envie dados pessoais reais ou confidenciais da empresa para serviços de IA sem autorização. A massa de

teste deve preferencialmente ser sintética/fictícia. A equipe é responsável por revisar a coerência dos dados gerados.

10. Fase 7 - Validação do banco

A implementação só é considerada concluída depois de testada. A equipe deve executar testes documentados e guardar evidências.

Teste	Ação	Resultado esperado	Evidência
PK	Tentar inserir ID duplicado	Operação rejeitada	Mensagem/print.
FK	Inserir venda para cliente inexistente	Operação rejeitada	Erro de integridade.
NOT NULL	Omitir campo obrigatório	Operação rejeitada	Erro.
JOIN	Consultar venda + cliente + itens	Dados integrados corretamente	Resultado.
UPDATE	Alterar preço/cadastro	Somente registro alvo alterado	SELECT antes/depois.
DELETE	Excluir registro permitido	Comportamento conforme regra	SELECT/erro esperado.

10.1 Teste de consistência conceitual

14. Teste da origem: para cada entidade, identifique o requisito/regra que a criou.
15. Teste da atributação: confira se cada atributo realmente pertence àquela entidade.
16. Teste de cardinalidade extrema: simule zero ocorrências, uma ocorrência e muitas ocorrências.
17. Teste de leitura inversa: leia cada relacionamento nos dois sentidos.
18. Teste de rastreabilidade: verifique se os requisitos estão representados no modelo e se o modelo não inventa funcionalidades.

11. Fase 8 - Proposta de arquitetura analítica para BI e IA

A equipe não precisa construir uma plataforma completa de BI ou IA. Deve propor, de forma técnica e coerente, como os dados operacionais modelados poderiam alimentar análises futuras.

11.1 Perguntas que a proposta deve responder

- Quais dados do ERP possuem valor analítico?
- Quais indicadores (KPIs) a gestão deveria acompanhar?
- Quais tabelas seriam fontes para relatórios ou dashboards?
- Seria necessário criar uma camada analítica, data mart ou data warehouse no futuro?
- Que tipo de aplicação de IA poderia utilizar esses dados?
- Quais cuidados de qualidade, privacidade, segurança e governança seriam necessários?

11.2 Exemplo de fluxo analítico

```
BANCO OPERACIONAL DO ERP
        ↓
ETL / ELT ou processo de preparação
        ↓
CAMADA ANALÍTICA / DATA MART
        ↓
DASHBOARDS / BI
        ↓
ANÁLISES / PREVISÕES / IA
```

11.3 Exemplos de BI e IA

- BI: faturamento, margem, estoque, produtos mais vendidos e desempenho por período.
- IA: previsão de demanda, recomendação de produtos, identificação de padrões de compra ou risco de ruptura de estoque.

As propostas de IA devem ser tratadas como possibilidades futuras. Não invente que o modelo operacional atual já resolve automaticamente problemas de IA.

12. Entrega 1 - Relatório Técnico Final

O relatório técnico deverá consolidar o projeto do início ao fim. Estrutura mínima recomendada:

Seção	Conteúdo mínimo	Evidência esperada
1. Introdução	Contexto, objetivo e escopo do ERP.	Descrição clara.
2. Empresa	Caracterização, processos e dores.	Cenário real/documentado.
3. Requisitos	RF, RNF e regras de negócio.	Itens numerados.
4. Modelo conceitual	MER/DER e dicionário de dados.	Diagrama legível.
5. Normalização	1FN, 2FN e 3FN.	Processo documentado.
6. Modelo lógico	Tabelas, PK, FK, restrições.	Esquema lógico.
7. Implementação	Decisões de SGBD e script.	Trechos/explicações.
8. Testes	Plano e evidências.	Resultados.
9. BI/IA	Arquitetura analítica proposta.	Diagrama e justificativa.
10. Decisões técnicas	Justificativas e trade-offs.	Memorial técnico.
11. Conclusão	Resultados, limitações e próximos passos.	Síntese.

13. Entrega 2 - Modelo Lógico Relacional

O documento/diagrama lógico deve apresentar claramente:

- Tabelas.
- Atributos.
- Tipos lógicos de dados.
- Chaves primárias.
- Chaves estrangeiras.
- Restrições de integridade.
- Cardinalidades convertidas.
- Entidades associativas oriundas de relacionamentos N:N.

Não confundir DER conceitual com modelo lógico
No DER conceitual, o foco é representar o domínio. No modelo lógico, aparecem explicitamente tabelas e FKs. Não antecipe FKs como atributos no DER conceitual apenas porque elas existirão posteriormente no banco.

14. Entrega 3 - Script SQL completo

O script final deverá conter, no mínimo:

- CREATE DATABASE.
- CREATE TABLE.
- Constraints.
- INSERT INTO.

- SELECT com JOIN.
- UPDATE.
- DELETE.
- Consultas gerenciais.
- Consultas de auditoria.

14.1 Organização recomendada dos arquivos SQL

```
/sql
  01_create_database.sql
  02_create_tables.sql
  03_constraints_indexes.sql
  04_insert_data.sql
  05_queries_operacionais.sql
  06_queries_gerenciais.sql
  07_queries_auditoria.sql
```

15. Entrega 4 - Massa de Dados

Inclua registros suficientes para validar os processos empresariais. A massa de dados deve acompanhar a estrutura do banco e permitir reproduzir os testes.

- Arquivo SQL de INSERTs ou CSVs organizados por tabela.
- Descrição da estratégia de geração.
- Prompts utilizados em LLMs, quando houver.
- Cuidados adotados para não utilizar dados pessoais reais desnecessariamente.
- Validações realizadas após a geração.

16. Entrega 5 - Repositório GitHub

O GitHub deverá evidenciar organização, documentação e desenvolvimento colaborativo. Não deve ser apenas um local onde os arquivos são enviados no último dia.

16.1 Estrutura sugerida

```
Projeto-ERP-ModelagemBD/
├── README.md
├── docs/
│   ├── Relatorio_Tecnico.pdf
│   ├── requisitos.md
│   ├── regras_negocio.md
│   └── prompts_massa_dados.md
├── diagramas/
│   ├── DER_conceitual.png
│   └── modelo_logico.png
├── sql/
│   ├── 01_create_database.sql
│   ├── 02_create_tables.sql
│   ├── 03_insert_data.sql
│   └── 04_queries.sql
├── dados/
│   └── arquivos_csv_ou_sql
└── evidencias/
    └── testes/
```

16.2 README.md mínimo

- Nome do projeto e equipe.
- Empresa/cenário selecionado.
- Problema empresarial.
- Escopo do ERP.
- Tecnologias utilizadas.
- Resumo do modelo de dados.
- Como recriar e testar o banco.
- Links para documentação e diagramas.
- Principais consultas gerenciais.

16.3 Commits

Os commits devem refletir evolução real do projeto. Evite um único commit final com todo o conteúdo.

```
Exemplos:
feat: adiciona requisitos funcionais do módulo de vendas
docs: inclui DER conceitual revisado
fix: corrige cardinalidade entre venda e pagamento
feat: cria tabelas produto e item_venda
test: adiciona consultas de auditoria
```

17. Entrega 6 - Apresentação Corporativa Final

A equipe deverá realizar um pitch técnico e executivo em tempo rigorosamente controlado. A apresentação deve ser visual, objetiva e orientada à defesa das decisões.

17.1 Conteúdo obrigatório

- Problema empresarial solucionado.
- Resumo dos processos e requisitos mais importantes.
- DER/modelo lógico e principais decisões.
- Demonstração de consultas gerenciais relevantes.
- Estrutura analítica proposta para BI/IA.
- Benefícios diretos para a tomada de decisão.

17.2 Regra visual

Slides
Utilize estética limpa e minimalista. Não leia longos blocos de texto. Prefira diagramas, fluxos, tabelas resumidas, consultas-chave e resultados visuais.

17.3 Preparação para arguição

- Todos os integrantes devem conhecer o modelo completo, não apenas sua parte.
- Treinem respostas para decisões de cardinalidade, PK/FK e normalização.
- Estejam preparados para explicar por que uma tabela existe.
- Saibam indicar qual requisito ou regra originou cada decisão importante.
- Conheçam limitações e alternativas que foram consideradas.

18. Sequência passo a passo de desenvolvimento

Etapa	Ação	Saída/Checkpoint

1. Formar a equipe	Definir integrantes, responsabilidades iniciais e forma de comunicação.	Equipe organizada
2. Escolher a empresa	Selecionar cenário real/documentado e registrar justificativa.	Empresa definida
3. Delimitar o escopo	Definir módulos incluídos e excluídos.	Escopo escrito
4. Mapear processos	Descrever como vendas, compras, estoque etc. funcionam.	Fluxos descritos
5. Identificar dores	Registrar problemas concretos e consequências.	Problemas concretos
6. Escrever RF/RNF/RN	Numerar requisitos e regras.	Catálogo numerado
7. Criar rastreabilidade	Ligar problemas a requisitos e regras.	Matriz
8. Identificar entidades	Derivar entidades a partir do negócio.	Lista de entidades
9. Definir atributos	Listar dados necessários e identificadores.	Atributos/IDs
10. Definir relacionamentos	Escrever as relações em frases.	Frases de relação
11. Validar cardinalidades	Ler em ambos os sentidos.	Cardinalidades
12. Criar DER	Produzir diagrama conceitual formal.	DER conceitual
13. Criar dicionário	Documentar entidades, atributos e semântica.	Dicionário
14. Normalizar	Registrar evolução 1FN, 2FN e 3FN.	1FN/2FN/3FN
15. Converter para lógico	Criar tabelas, PKs, FKs e restrições.	Modelo lógico
16. Definir SGBD	Justificar MySQL, PostgreSQL ou opção aprovada.	Justificativa
17. Implementar SQL	Criar estrutura e constraints.	DDL
18. Gerar massa	Popular banco e documentar prompts.	Dados de teste
19. Testar integridade	PK, FK, NOT NULL, UNIQUE, regras.	Evidências
20. Criar consultas	Operacionais, gerenciais e auditoria.	SQL de negócio
21. Propor BI/IA	Criar arquitetura analítica futura.	Arquitetura
22. Consolidar documentação	Relatório, diagramas, scripts e evidências.	Relatório final
23. Revisar GitHub	README, commits, organização.	Repositório final
24. Ensaiar pitch	Dividir falas, cronometrar e preparar arguição.	Apresentação pronta

19. Organização do trabalho em equipe

A divisão de tarefas é permitida e desejável, mas cada fase deve ser revisada em conjunto. A modelagem não pode ser fragmentada de forma que cada integrante produza tabelas independentes sem integração.

Papel sugerido	Responsabilidade principal	Atenção
Analista de negócio	Processos, dores, RF/RNF/RN	Não trabalhar isolado do modelador.
Modelador	DER, dicionário, cardinalidades	Validar tudo com requisitos.
Responsável SQL	Modelo lógico, DDL/DML, consultas	Não redesenhar o modelo sozinho.
Dados/Testes	Massa, testes e evidências	Garantir coerência e integridade.
Documentação/DevOps	Relatório, GitHub, README	Todos devem contribuir em commits.

20. Memorial de decisões técnicas

O relatório deverá registrar decisões que poderiam ter sido modeladas de mais de uma forma. Isso demonstra maturidade técnica e facilita a defesa do projeto.

Decisão	Alternativas	Escolha	Justificativa
Preço histórico	Guardar só em PRODUTO /	ITEM_VENDA	Preço da venda não pode mudar

	guardar também no item		quando o catálogo muda.
Cliente obrigatório	Permitir venda anônima / exigir cliente	Conforme regra do negócio	Basear-se no processo real.
Exclusão	DELETE físico / inativação	Conforme histórico necessário	Preservar rastreabilidade quando necessário.

21. Erros críticos a evitar

- Começar criando tabelas antes de entender o negócio.
- Inventar entidades apenas para aumentar o tamanho do DER.
- Copiar um ERP pronto sem demonstrar relação com a empresa escolhida.
- Confundir requisito com regra de negócio.
- Usar atributos vagos sem descrição no dicionário.
- Criar N:N e não tratar a relação no modelo lógico.
- Colocar FK explícita no modelo conceitual apenas por hábito de SQL.
- Normalizar apenas no texto sem mostrar o processo.
- Gerar grande massa de dados com IA e não validar consistência.
- Criar consultas que não respondem perguntas reais da empresa.
- Entregar diagramas ilegíveis.
- Fazer todos os commits no último dia.
- Colocar longos parágrafos nos slides e ler durante a apresentação.

22. Checklist final da equipe

☐ A empresa e o escopo estão claramente definidos.

☐ Há pelo menos três problemas operacionais concretos.

☐ RF, RNF e RN estão numerados e específicos.

☐ Existe rastreabilidade entre dores, requisitos, regras e modelo.

☐ Todas as entidades possuem justificativa.

☐ Atributos e identificadores foram documentados.

☐ Relacionamentos e cardinalidades foram validados nos dois sentidos.

☐ Relações N:N foram tratadas adequadamente.

☐ O DER conceitual está legível.

☐ O dicionário de dados cobre todo o DER.

☐ A normalização 1FN, 2FN e 3FN está documentada.

☐ O modelo lógico contém tabelas, PK, FK e restrições.

☐ O SQL recria o banco conforme o SGBD escolhido.

☐ Há INSERT, SELECT/JOIN, UPDATE e DELETE.

☐ Há consultas gerenciais e de auditoria.

☐ A massa de dados é coerente e suficiente para testes.

☐ Prompts de LLM usados para massa de dados estão documentados.

☐ Os testes de integridade possuem evidências.

☐ A proposta de BI/IA é coerente com os dados disponíveis.

☐ O GitHub está organizado e possui histórico de commits.

☐ O README explica como compreender e executar o projeto.

☐ A apresentação foi ensaiada e está dentro do tempo.

☐ Todos os integrantes conseguem defender as principais decisões.

23. Modelo de sumário do Relatório Técnico Final

```
CAPA
SUMÁRIO
1. INTRODUÇÃO
2. CARACTERIZAÇÃO DA EMPRESA E ESCOPO
3. PROCESSOS E PROBLEMAS OPERACIONAIS
4. REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS
5. REGRAS DE NEGÓCIO
6. MATRIZ DE RASTREABILIDADE
7. MODELO CONCEITUAL - MER/DER
8. DICIONÁRIO DE DADOS
9. NORMALIZAÇÃO - 1FN, 2FN E 3FN
10. MODELO LÓGICO RELACIONAL
11. IMPLEMENTAÇÃO E SCRIPT SQL
12. MASSA DE DADOS E PROMPTS UTILIZADOS
13. TESTES E VALIDAÇÕES
14. CONSULTAS GERENCIAIS E DE AUDITORIA
15. PROPOSTA DE ARQUITETURA PARA BI E IA
16. MEMORIAL DE DECISÕES TÉCNICAS
17. CONCLUSÃO
REFERÊNCIAS
APÊNDICES/ANEXOS
```

24. Orientação final

O valor principal do projeto não está na quantidade de tabelas nem no tamanho do script. O objetivo é demonstrar que a equipe consegue transformar um problema empresarial em uma solução de dados coerente, rastreável, normalizada, implementável e útil para operação e tomada de decisão.

Pergunta final de auditoria
Escolha qualquer tabela, campo, relacionamento, constraint ou consulta do projeto e pergunte: "De onde veio esta decisão e que problema/requisito ela atende?" Se a equipe não conseguir responder, o artefato precisa ser revisto.